

# Factory Telemetry & OEE Monitor

Demonstracja działania aplikacji w chmurze Microsoft Azure

---

**Autor:** Dawid B.

**Data:** 1 czerwca 2026

**Środowisko:** Microsoft Azure — App Service (Linux/.NET 9), Azure SQL, IoT Hub

**Adres aplikacji:** <https://app-factorytel-dev-hnzy6.azurewebsites.net>

**Repozytorium:** [https://github.com/dawidbis/loT\\_Production\\_Monitor](https://github.com/dawidbis/loT_Production_Monitor)

*Dokument roboczy: w miejscach oznaczonych ramką wklej zrzuty ekranu (Wstaw → Obrazy, lub przeciągnij plik na ramkę).*

## 1. O projekcie

Factory Telemetry & OEE Monitor to lekki system backendowy (API), który symuluje odbieranie danych z sensorów na linii produkcyjnej — np. zrobotyzowanego gniazda spawalniczego. Aplikacja przyjmuje sygnały JSON o statusie maszyny (stan, temperatura, liczba wyprodukowanych i wadliwych sztuk), zapisuje je w bazie danych i udostępnia endpoint obliczający wskaźnik OEE.

OEE (Overall Equipment Effectiveness / Całkowita Efektywność Wyposażenia) to kluczowy wskaźnik w każdym zakładzie produkcyjnym, liczony jako:

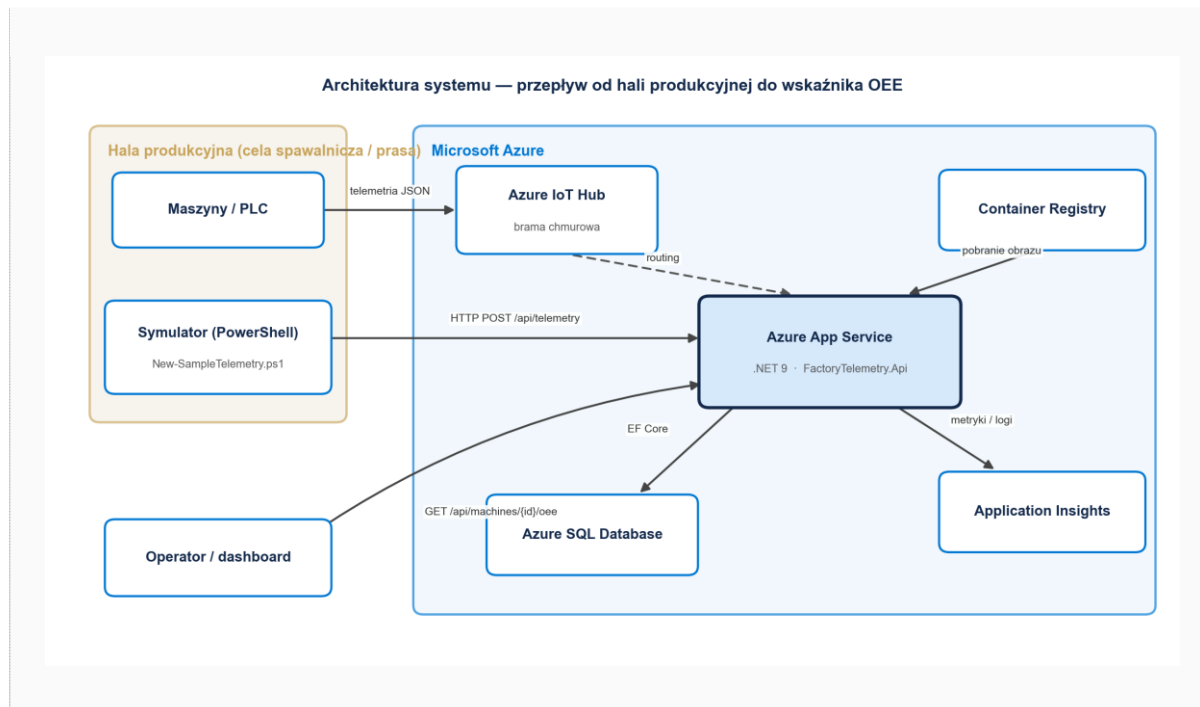
$$\text{OEE} = \text{Dostępność} \times \text{Wydajność} \times \text{Jakość}$$

Projekt powstał jako portfolio DevOps/Cloud: sama aplikacja jest celowo niewielka, a nacisk położono na automatyzację wokół niej — Infrastructure as Code, CI/CD, skrypty, testy i wdrożenie do chmury Microsoft.

## 2. Architektura rozwiązania

Przepływ danych od hali produkcyjnej do wskaźnika OEE:

- Maszyny / symulator (PowerShell) wysyłają telemetrię HTTP (JSON) do API.
- API (.NET 9 na Azure App Service) waliduje i zapisuje odczyty do Azure SQL (przez Entity Framework Core).
- Endpoint OEE pobiera odczyty z okna czasowego i liczy Dostępność × Wydajność × Jakość.
- Azure IoT Hub pełni rolę bramy chmurowej; Application Insights zbiera telemetrię działania.



Rys. 1 — Architektura rozwiązania

## 3. Stos technologiczny (5 warstw)

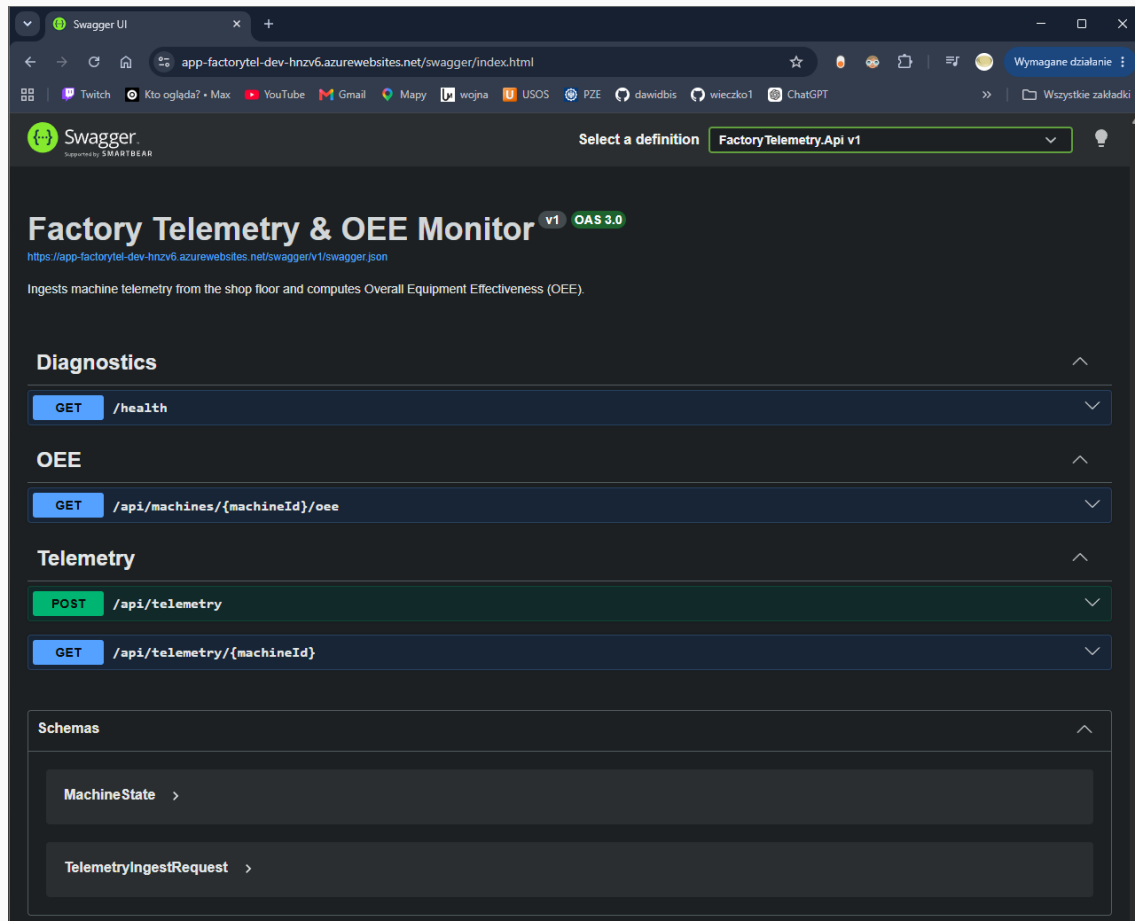
| Warstwa                       | Technologie                                                                   |
|-------------------------------|-------------------------------------------------------------------------------|
| 1. Infrastruktura (IaC)       | Terraform (HCL) + AzureRM: App Service, Azure SQL, IoT Hub, ACR, App Insights |
| 2. CI/CD                      | GitHub Actions (build + testy + lint), Azure Pipelines (YAML)                 |
| 3. Skrypty                    | PowerShell 7, Pester (testy), PSScriptAnalyzer (analiza statyczna)            |
| 4. Aplikacja                  | C# / .NET 9 Minimal API, EF Core (migracje), Serilog, Swagger / OpenAPI       |
| 5. Zarządzanie i dokumentacja | Backlog produktu (User Stories), architektura, runbook                        |

## 4. Demonstracja działania (na żywo, Azure)

Wszystkie poniższe operacje wykonano na działającej instancji w chmurze Azure, pod adresem `app-factorytel-dev-hnztv6.azurewebsites.net`, z użyciem interaktywnej dokumentacji Swagger / OpenAPI.

### 4.1. Interaktywna dokumentacja API (Swagger / OpenAPI)

API automatycznie generuje dokumentację zgodną z OpenAPI 3.0, z endpointami pogrupowanymi w sekcje: Diagnostics, Telemetry oraz OEE.

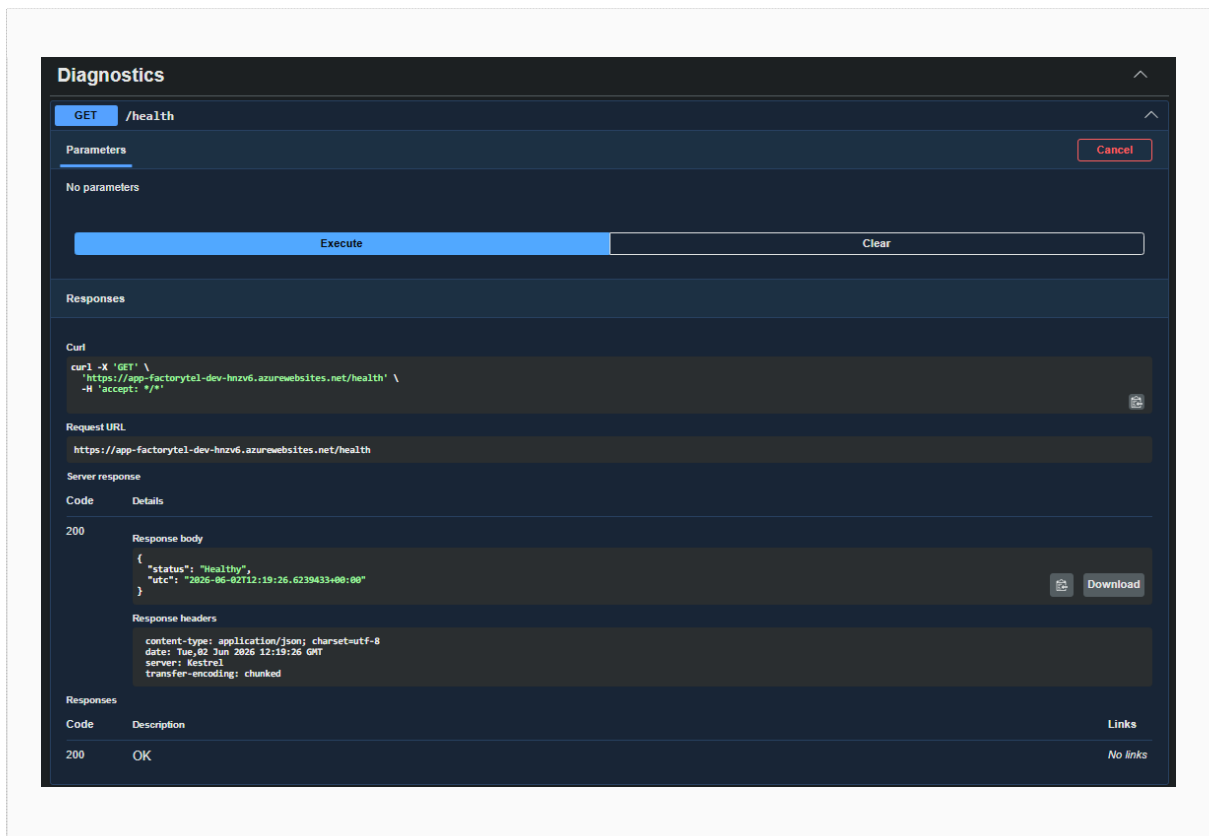


Rys. 2 — Interaktywna dokumentacja API (OpenAPI 3.0)

### 4.2. Kontrola stanu usługi (Health check)

Zapytanie GET `/health` potwierdza, że usługa działa. Odpowiedź:

```
GET /health -> 200 OK
{
  "status": "Healthy",
  "utc": "2026-06-01T21:03:02Z"
}
```



Rys. 3 — Kontrola stanu usługi

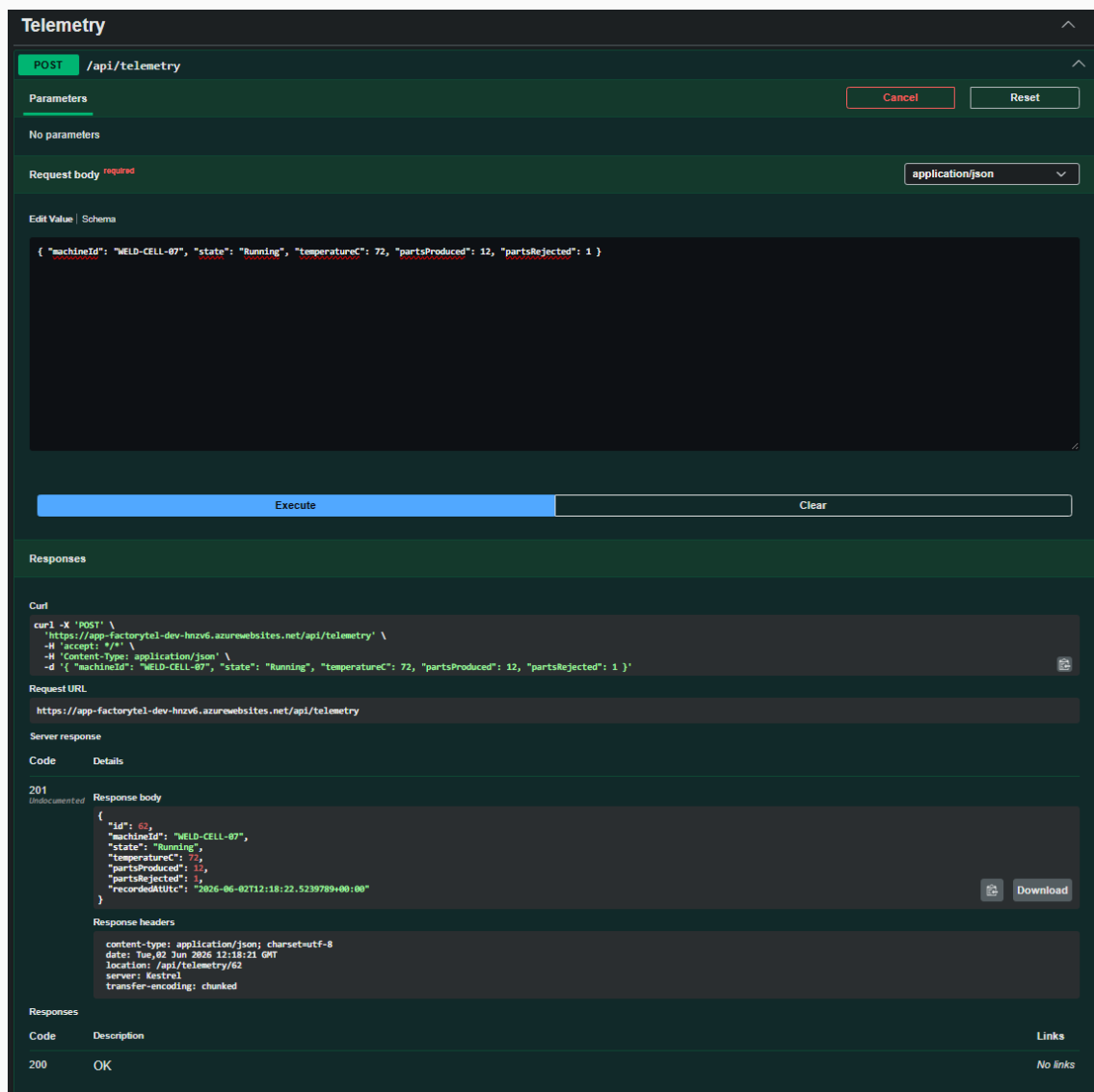
### 4.3. Przyjęcie sygnału z maszyny (POST /api/telemetry)

Wysłanie pojedynczego odczytu telemetry. Treść żądania:

```
POST /api/telemetry
{
  "machineId": "WELD-CELL-07",
  "state": "Running",
  "temperatureC": 72.5,
  "partsProduced": 10,
  "partsRejected": 1
}
```

Odpowiedź 201 Created — rekord został zapisany w Azure SQL, a baza nadała mu identyfikator:

```
201 Created (Location: /api/telemetry/66)
{
  "id": 66,
  "machineId": "WELD-CELL-07",
  "state": "Running",
  "temperatureC": 72.5,
  "partsProduced": 10,
  "partsRejected": 1,
  "recordedAtUtc": "2026-06-01T21:06:23Z"
}
```

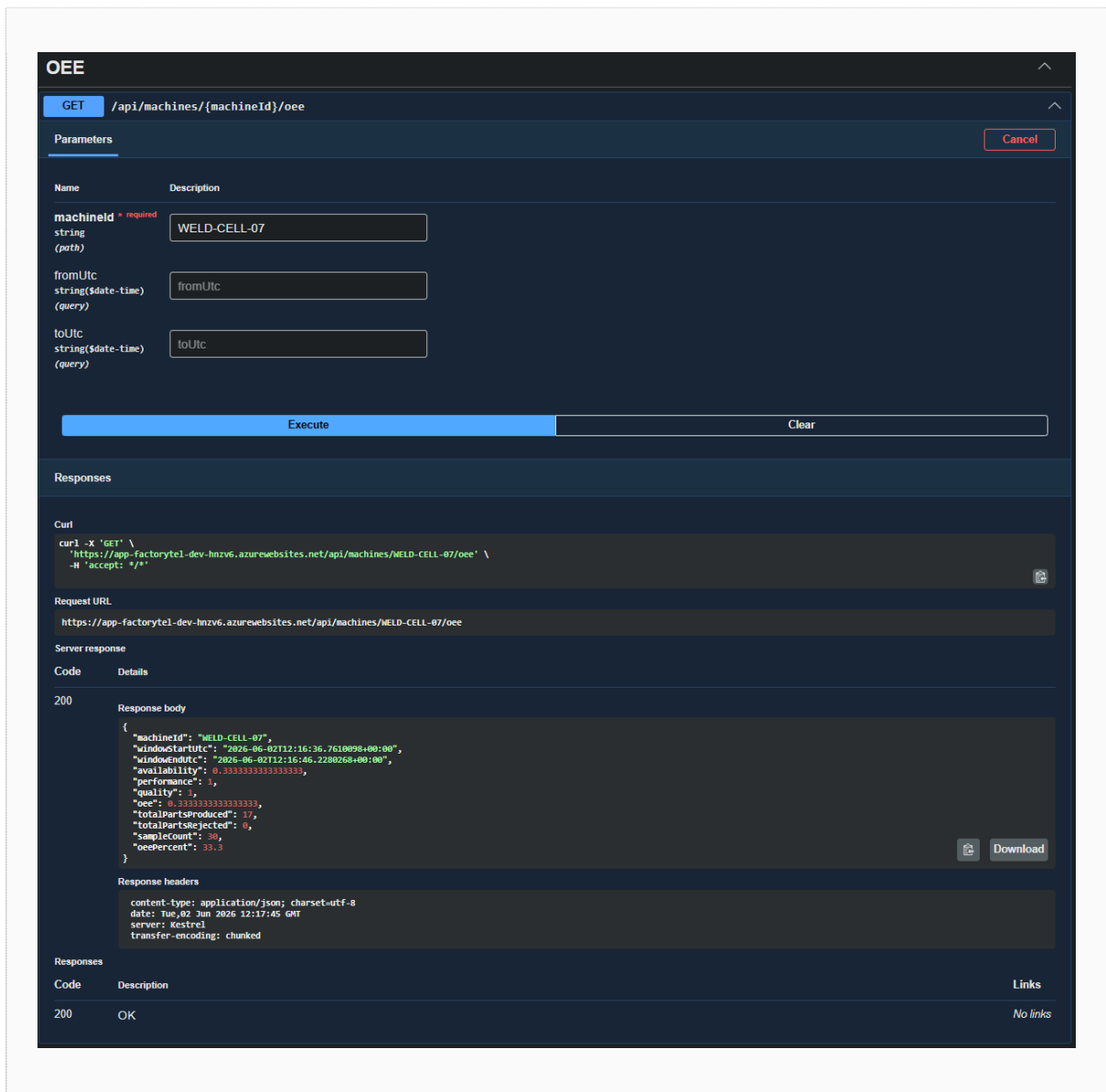


Rys. 4 — Przyjęcie i zapis sygnału z maszyny

#### 4.4. Obliczenie wskaźnika OEE (GET /api/machines/{machineId}/oee)

Dla maszyny WELD-CELL-07 API policzyło OEE na podstawie 33 odczytów zebranych w oknie czasowym. Odpowiedź:

```
GET /api/machines/WELD-CELL-07/oee -> 200 OK
{
  "machineId": "WELD-CELL-07",
  "availability": 0.6296,
  "performance": 1.0,
  "quality": 0.9048,
  "oee": 0.5697,
  "oeePercent": 57,
  "totalPartsProduced": 63,
  "totalPartsRejected": 6,
  "sampleCount": 33
}
```



Rys. 5 — Obliczenie wskaźnika OEE z danych zapisanych w Azure SQL

## 5. Wyniki — wskaźnik OEE

Wartości obliczone na żywo dla maszyny WELD-CELL-07 (33 próbki, 63 sztuki wyprodukowane, 6 odrzuconych):

| Wskaźnik             | Wartość | Definicja                                          |
|----------------------|---------|----------------------------------------------------|
| <b>Dostępność</b>    | 63,0 %  | Czas pracy / planowany czas produkcji              |
| <b>Wydajność</b>     | 100,0 % | (Nominalny czas cyklu × liczba sztuk) / czas pracy |
| <b>Jakość</b>        | 90,5 %  | Dobre sztuki / wszystkie sztuki                    |
| <b>OEE (łącznie)</b> | 57 %    | Dostępność × Wydajność × Jakość                    |

Wynik 57% jest realistyczny dla linii produkcyjnej (poziom „world-class” to ok. 85%); głównym ograniczeniem jest tu Dostępność, co od razu wskazuje obszar do poprawy.

## 6. Infrastruktura w Azure

Wszystkie zasoby utworzone deklaratywnie przez Terraform, w grupie zasobów rg-factorytel-dev:

- App Service (Linux, .NET 9): app-factorytel-dev-hnzv6 — region Poland Central
- Azure SQL Database: sql-factorytel-dev-hnzv6 / baza sqldb-telemetry
- Azure IoT Hub: iot-factorytel-dev-hnzv6 — region Germany West Central
- Azure Container Registry oraz Application Insights + Log Analytics

```
Wdrozenie zakonczone.

[Enter] - menu Azure:

--- Zarzadzanie infrastruktura Azure ---
[1] Provisioning infrastruktury (terraform apply [+ deploy])
[2] Wdraz / aktualizuj aplikacje (publish + az webapp deploy)
[3] Wlacz aplikacje (az webapp start)
[4] Wylacz aplikacje (az webapp stop)
[5] USUN wszystkie zasoby (terraform destroy)
[6] Pokaz stan zasobow Azure
[7] Zaloguj do Azure / ustaw haslo (az login)
[0] Powrot do menu glownego

Wybierz opcje Azure: 6

Grupa zasobow: rg-factorytel-dev
Zasoby w grupie:
Nazwa                                     Typ                                     Region
-----
sql-factorytel-dev-hnzv6                 Microsoft.Sql/servers                 polandcentral
plan-factorytel-dev                     Microsoft.Web/serverFarms             polandcentral
acr-factoryteldevhnzv6                   Microsoft.ContainerRegistry/registries polandcentral
iot-factorytel-dev-hnzv6                 Microsoft.Devices/IotHubs             germanywestcentral
log-factorytel-dev                       Microsoft.OperationalInsights/workspaces polandcentral
appi-factorytel-dev                      Microsoft.Insights/components         polandcentral
sql-factorytel-dev-hnzv6/master           Microsoft.Sql/servers/databases        polandcentral
sql-factorytel-dev-hnzv6/sqldb-telemetry Microsoft.Sql/servers/databases        polandcentral
app-factorytel-dev-hnzv6                 Microsoft.Web/sites                   polandcentral
App Service 'app-factorytel-dev-hnzv6' stan: Running

[Enter] - menu Azure: |
```

Rys. 6 — Zasoby utworzone przez Terraform

## 7. Automatyzacja (IaC + CI/CD)

- Terraform — cała infrastruktura jako kod, stan zdalny w Azure Storage; weryfikacja terraform fmt + validate.
- GitHub Actions — przy każdym push uruchamiane są testy .NET (xUnit), testy PowerShell (Pester), PSScriptAnalyzer i walidacja Terraform (status: passing).
- EF Core Migrations — schemat bazy tworzony automatycznie przy starcie aplikacji.
- PowerShell — symulator telemetry hali produkcyjnej oraz raport OEE; jakość pilnowana przez Pester i PSScriptAnalyzer.

## 8. Konsola operatora — provisioning i obsługa (na żywo)

Całe środowisko Azure obsługiwane jest z jednej konsoli operatora (PowerShell): od logowania i provisioningu infrastruktury (terraform apply), przez wdrożenie aplikacji i włączanie/wyłączanie usługi, po jej całkowite usunięcie (terraform destroy) w celu kontroli kosztów. Poniżej kolejne kroki uruchomienia i obsługi środowiska na żywo.

```
C:\Windows\system32\cmd.e: X + v

=====
Factory Telemetry & OEE Monitor - konsola operatora
=====

Srodowisko : Azure
Adres       : https://app-factorytel-dev-hnzv6.azurewebsites.net
Status API  : OFFLINE
Maszyny     : WELD-CELL-07, PRESS-12
=====

[1] Sprawdz stan API (health)
[2] Wysluj telemetrie (symulator hali)
[3] Raport OEE
[4] Monitor na zywo (auto-odswiezanie)
[5] Ostatnie odczyty maszyny
[6] Otworz Swagger w przegladarce
[7] Zmien srodowisko (Azure / Local / wlasny URL)
[8] Bramka jakosci (PSScriptAnalyzer + Pester)
[9] Zarzadzanie Azure (provision / start / stop / usun)
[0] Wyjscie

Wybierz opcje: 9

--- Zarzadzanie infrastruktura Azure ---
[1] Provisioning infrastruktury (terraform apply [+ deploy])
[2] Wdroz / aktualizuj aplikacje (publish + az webapp deploy)
[3] Wlacz aplikacje (az webapp start)
[4] Wylacz aplikacje (az webapp stop)
[5] USUN wszystkie zasoby (terraform destroy)
[6] Pokaz stan zasobow Azure
[7] Zaloguj do Azure / ustaw haslo (az login)
[0] Powrot do menu glownego

Wybierz opcje Azure: 7

Zalogowano: db310378@student.polsl.pl (subskrypcja: Azure for Students)
Haslo administratora SQL (min. 8 znakow: duze+male+cyfra): *****
Ustawiono $env:SQL_ADMIN_PASSWORD dla tej sesji.

Gotowe. Uruchom konsole -> [9] Zarzadzanie Azure -> [1] Provisioning.

[Enter] - menu Azure: |
```

Rys. 7 — Konsola operatora: stan początkowy (status OFFLINE) oraz logowanie do Azure (az login, subskrypcja „Azure for Students”) i ustawienie hasła SQL dla sesji.

```
C:\Windows\system32\cmd.e: X + v

=====
Factory Telemetry & OEE Monitor - konsola operatora
=====

Srodowisko : Azure (tf)
Adres      : https://app-factorytel-dev-hnzv6.azurewebsites.net
Status API : ONLINE
Maszyny    : WELD-CELL-07, PRESS-12
-----

[1] Sprawdź stan API (health)
[2] Wyslij telemetrie (symulator hali)
[3] Raport OEE
[4] Monitor na zywo (auto-odswiezanie)
[5] Ostatnie odczyty maszyny
[6] Otworz Swagger w przegladarce
[7] Zmien srodowisko (Azure / Local / własny URL)
[8] Bramka jakosci (PSScriptAnalyzer + Pester)
[9] Zarządzanie Azure (provision / start / stop / usun)
[0] Wyjście

Wybierz opcje: |
```

Rys. 8 — Po provisioningu: status API ONLINE i menu operatora z opcją „[9] Zarządzanie Azure (provision / start / stop / usun)”.

C:\Windows\system32\cmd.e: X + v

## Factory Telemetry & OEE Monitor - konsola operatora

Srodowisko : Azure (tf)  
Adres : <https://app-factorytel-dev-hnqv6.azurewebsites.net>  
Status API : **ONLINE**  
Maszyny : WELD-CELL-07, PRESS-12

- [1] Sprawdz stan API (health)
- [2] Wysluj telemetry (symulator hali)
- [3] Raport OEE
- [4] Monitor na zywo (auto-odswiezanie)
- [5] Ostatnie odczyty maszyny
- [6] Otworz Swagger w przegladarce
- [7] Zmien srodowisko (Azure / Local / wlasny URL)
- [8] Bramka jakosci (PSScriptAnalyzer + Pester)
- [9] Zarzadzanie Azure (provision / start / stop / usun)
- [0] Wyjscie

Wybierz opcje: 2

Ile probek na maszynie? [30]:  
Odstep miedzy cyklami (ms)? [150]:

Streaming 30 samples per machine to <https://app-factorytel-dev-hnqv6.azurewebsites.net> ...

|         |              |             |            |            |
|---------|--------------|-------------|------------|------------|
| [ 1/30] | WELD-CELL-07 | Idle        | produced=0 | rejected=0 |
| [ 1/30] | PRESS-12     | Running     | produced=1 | rejected=0 |
| [ 2/30] | WELD-CELL-07 | PlannedStop | produced=0 | rejected=0 |
| [ 2/30] | PRESS-12     | PlannedStop | produced=0 | rejected=0 |
| [ 3/30] | WELD-CELL-07 | Down        | produced=0 | rejected=0 |
| [ 3/30] | PRESS-12     | PlannedStop | produced=0 | rejected=0 |
| [ 4/30] | WELD-CELL-07 | Running     | produced=2 | rejected=0 |
| [ 4/30] | PRESS-12     | Running     | produced=2 | rejected=0 |
| [ 5/30] | WELD-CELL-07 | Idle        | produced=0 | rejected=0 |
| [ 5/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [ 6/30] | WELD-CELL-07 | Down        | produced=0 | rejected=0 |
| [ 6/30] | PRESS-12     | Running     | produced=3 | rejected=0 |
| [ 7/30] | WELD-CELL-07 | PlannedStop | produced=0 | rejected=0 |
| [ 7/30] | PRESS-12     | Idle        | produced=0 | rejected=0 |
| [ 8/30] | WELD-CELL-07 | Running     | produced=3 | rejected=0 |
| [ 8/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [ 9/30] | WELD-CELL-07 | Idle        | produced=0 | rejected=0 |
| [ 9/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [10/30] | WELD-CELL-07 | Down        | produced=0 | rejected=0 |
| [10/30] | PRESS-12     | Running     | produced=3 | rejected=0 |
| [11/30] | WELD-CELL-07 | PlannedStop | produced=0 | rejected=0 |
| [11/30] | PRESS-12     | Running     | produced=3 | rejected=0 |
| [12/30] | WELD-CELL-07 | Running     | produced=2 | rejected=0 |
| [12/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [13/30] | WELD-CELL-07 | Running     | produced=1 | rejected=0 |
| [13/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [14/30] | WELD-CELL-07 | PlannedStop | produced=0 | rejected=0 |
| [14/30] | PRESS-12     | Running     | produced=5 | rejected=0 |
| [15/30] | WELD-CELL-07 | Down        | produced=0 | rejected=0 |
| [15/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [16/30] | WELD-CELL-07 | Running     | produced=2 | rejected=0 |
| [16/30] | PRESS-12     | Idle        | produced=0 | rejected=0 |
| [17/30] | WELD-CELL-07 | PlannedStop | produced=0 | rejected=0 |
| [17/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [18/30] | WELD-CELL-07 | Down        | produced=0 | rejected=0 |
| [18/30] | PRESS-12     | Running     | produced=4 | rejected=0 |
| [19/30] | WELD-CELL-07 | Down        | produced=0 | rejected=0 |
| [19/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [20/30] | WELD-CELL-07 | Idle        | produced=0 | rejected=0 |
| [20/30] | PRESS-12     | Running     | produced=4 | rejected=0 |
| [21/30] | WELD-CELL-07 | Idle        | produced=0 | rejected=0 |
| [21/30] | PRESS-12     | Down        | produced=0 | rejected=0 |
| [22/30] | WELD-CELL-07 | PlannedStop | produced=0 | rejected=0 |
| [22/30] | PRESS-12     | Running     | produced=1 | rejected=0 |
| [23/30] | WELD-CELL-07 | Running     | produced=3 | rejected=0 |
| [23/30] | PRESS-12     | Running     | produced=4 | rejected=0 |

Rys. 9 — Symulator hali produkcyjnej ([2]): strumień telemetry maszyn wysyłany do API i zapisywany w Azure SQL.

```
C:\Windows\system32\cmd.e: X + v

=====
Factory Telemetry & OEE Monitor - konsola operatora
=====

Srodowisko : Azure (tf)
Adres      : https://app-factorytel-dev-hnzv6.azurewebsites.net
Status API : ONLINE
Maszyny    : WELD-CELL-07, PRESS-12
-----

[1] Sprawdź stan API (health)
[2] Wyslij telemetry (symulator hali)
[3] Raport OEE
[4] Monitor na zywo (auto-odswiezanie)
[5] Ostatnie odczyty maszyny
[6] Otworz Swagger w przegladarce
[7] Zmien srodowisko (Azure / Local / własny URL)
[8] Bramka jakosci (PSScriptAnalyzer + Pester)
[9] Zarzadzanie Azure (provision / start / stop / usun)
[0] Wyjscie

Wybierz opcje: 3

OEE report for WELD-CELL-07
Availability : 33,3%
Performance : 100,0%
Quality      : 100,0%
-----
OEE          : 33,3%
Samples      : 30 | Produced: 17 Rejected: 0

OEE report for PRESS-12
Availability : 52,0%
Performance : 100,0%
Quality      : 100,0%
-----
OEE          : 52,0%
Samples      : 30 | Produced: 39 Rejected: 0

[Enter] - powrot do menu: |
```

Rys. 10 — Raport OEE wyliczony w konsoli ([3]) dla maszyn WELD-CELL-07 i PRESS-12.

## 9. Podsumowanie

Projekt pokazuje pełny, działający łańcuch DevOps na stacku Microsoft: od kodu aplikacji (.NET), przez Infrastructure as Code (Terraform) i CI/CD (GitHub Actions / Azure Pipelines), po wdrożenie i działanie w chmurze Azure z trwałym zapisem do Azure SQL. Aplikacja realizuje konkretny przypadek przemysłowy — monitoring telemetry maszyn i obliczanie OEE.